

데이터구조2, 클래스

CSED101 프로그래밍과 문제해결

Student ID: 20240505

Name: 공현성

POVIS ID: hyunseong

Phone Number: 010-9649-3020

4월 수업 일정

04/05 (토)

04/12 (토)

04/19 (토) (TODAY)

04/26 (토)

Content

1. 앞으로의 일정
2. 데이터구조2
3. 클래스

03/08	OT, 파이썬 설치, 개념 복습
03/15	입출력, 변수, 타입
03/22	연산자, 표현식, 흐름제어
03/29	데이터 구조, 파일 입출력
04/05	중간 모의고사 문제풀이
04/12	앞으로의 일정, 데이터구조2
04/19	클래스 (TODAY)
04/26	정렬과 검색
05/03	재귀
05/10	그래픽 프로그래밍, 퀴즈 대비
05/17	Numpy, Pandas
05/24	기말 모의고사 & 모의고사 문제풀이

앞으로의 일정

기말고사: 06/02 ~ 06/06

준비기간: 4월 중반 ~ 5월 후반

SUN	MON	TUE	WED	THU	FRI	SAT
13	14	15	16	17	18	19 수업
20	21	22	23	24	25	26 수업
27	28	29	30			

SUN	MON	TUE	WED	THU	FRI	SAT
				1	2	3 수업
4	5	6	7	8	9	10 수업
11	12	13	14	15	16	17 수업
18	19	20	21	22	23	24 기말준비

데이터구조2

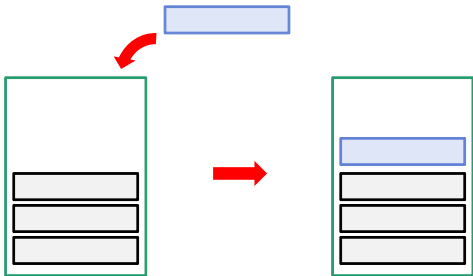
`collections`는 파이썬의 내장 모듈로 다양한 데이터 구조를 지원한다.

- ① 스택(`stack`)
- ② 큐(`queue`)
- ③ 디큐(`deque`)
- ④ 순서딕셔너리(`ordered dict`)
- ⑤ 계수기(`counter`)

데이터구조2 - 스택

스택(stack)은 First-In Last-Out 구조를 가진 데이터 구조이다.

파이썬에서는 리스트 자체가 스택으로서 사용될 수 있도록 기능을 제공하나, collections 모듈을 이용하면 이를 더 명확하게 쓸 수 있다.



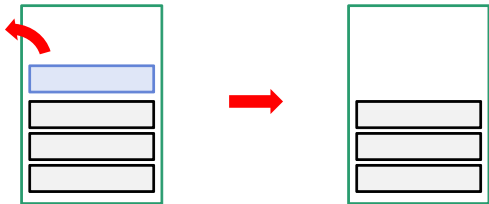
스택의 값 넣기는 다음과 같이 이루어진다.

데이터구조2 - 스택

함수 호출은 스택으로 관리된다.

내가 함수 내에서 다른 함수를 실행하면 현재 실행 위치가 스택에 하나씩 저장된다. 내부 함수가 값을 return하면 스택에서 원래 실행 위치를 찾아서 꺼낸다.

이렇게 하는 이유는, 함수가 여러 회 중첩되어 있을 때도 올바르게 위치를 찾기 위함이다.

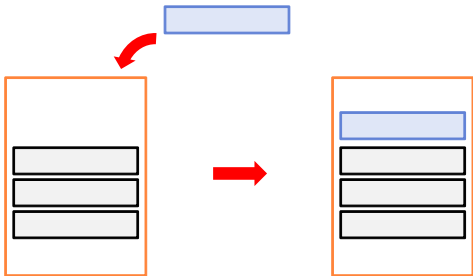


스택의 값 꺼내기는
다음과 같이 이루어진다.

데이터구조2 - 큐

큐(queue)는 First-In First-Out 구조를 가진 데이터 구조이다.

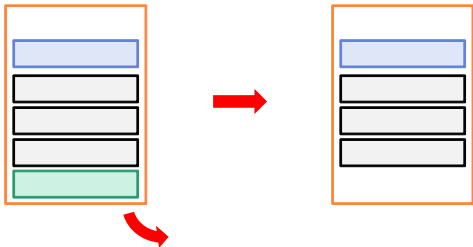
스택과 마찬가지로 파이썬에서 리스트를 큐로 사용될 수 있도록 기능을 제공한다.



큐의 값 넣기는 다음과 같이 이루어진다.

데이터구조2 - 큐

큐는 은행 번호 대기표에 응용해볼 수 있다. 빠른 번호를 뽑은 사람이 먼저 서비스를 제공받아야 한다.



큐의 값 꺼내기는 다음과 같이 이루어진다.

데이터구조2 - 나머지

- ① 디큐(**deque**): 양방향 큐이다. 즉, 양쪽 방향으로 넣고 빼기가 가능하다.
- ② 순서딕셔너리(**ordered dict**): 순서가 주어진 딕셔너리다. 원래 딕셔너리에서는 순서가 없다는 사실을 기억하자.
- ③ 계수기(**counter**): 리스트의 각 데이터의 개수를 딕셔너리 {데이터:개수,...}로 반환한다.

클래스 - 객체지향

프로그래밍을 하는 방법은 여러 가지가 있다. 그 중 하나는 객체지향이다. 기존의 우리는 절차지향으로 프로그래밍해왔었다.

- ① 절차지향: 함수 호출 기반
- ② 객체지향: 객체 기반
- ③ 함수지향: 표현식 기반

객체지향의 메인 컨셉은 우리가 사물을 보는 것과 비슷하게 코딩하자는 것이다.

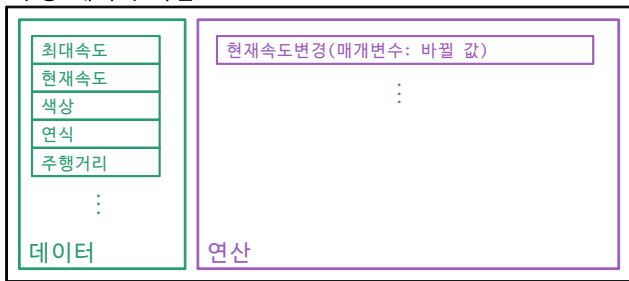
가령, 우리가 자동차 추돌 사고를 볼 땐, 자동차라고 하는 두 객체가 서로 상호작용 (물리력을 주고 받음)하는 것이라고 생각한다.

클래스 - ADT

추상 데이터 타입(**Abstract Data Type, ADT**)은 데이터 + 연산으로 구성된 수학적 모델로, “객체”라고 하는 대상을 다루는데 필수적이다.

추상화: 필요없는 것을 다 지워나가는 과정
(ex) 자동차에 꼭 필요한 건 있을까?

추상 데이터 타입



클래스 - 클래스란?

클래스(**class**)는 ADT를 코드로 구현하기 위한 구체적인 형식을 갖춘 대상 클래스는 하나의 틀(frame)이다. 비유하자면 쿠키 틀 같은거.

객체(**object**)는 클래스를 사용하여 만든 실제 요소이다. 비유하자면 쿠키.
인스턴스(**instance**)는 보다 구체적인 객체의 예시를 나타낼 때 사용한다. 보통 객체랑 혼용되어 쓴다.

파이썬에서 클래스는 하나의 커스텀 자료형을 만들 때 사용된다.

클래스 : 객체 = 사람 : 김연아 라고 하면, 김연아는 객체이기도 하지만 대한민국에 김연아라는 이름을 가진 사람이 많겠죠?
그중에서 "피겨여왕 김연아"라고 구체적으로 말하면, 객체의 실제 예를 가리키니까 이 때는 인스턴스라고 부를 수 있겠죠.

<https://www.infllearn.com/community/questions/890344/객체와-인스턴스의-차이>

클래스 - 클래스 구성요소

클래스에서 데이터를 선언하는 위치를 필드(**field**)라고한다.

클래스에서 기능은 전부 함수로 구현되며, 이렇게 클래스 내부에서 구현된 함수를 메서드(**method**)라고 부른다.

모든 메서드는 내부 인스턴스의 요소들을 불러올 수 있게 self 매개변수를 갖는다.

클래스 - 생성자, 소멸자

클래스 메서드 중에서 특별한 여러 메서드들이 있다.

- ① 생성자(**constructor**): 인스턴스 생성 시에 무조건적으로 호출 되는 특수 메서드이다.
- ② 소멸자(**destructor**): 인스턴스 소멸 시에 무조건적으로 호출되는 특수 메서드이다.

클래스 - 클래스의 특징

클래스의 특징은 여러가지가 있다.

- ① 캡슐화(encapsulation): 객체 내부 구현을 감춘다.
- ② 다형성(polymorphism): 클래스를 유연하게 만들어 재사용성을 높인다.
- ③ 상속(inheritance): 부모 클래스의 모든 요소를 물려받은 자식 클래스를 만든다. 이렇게 물려받은 요소를 일부 수정하거나, 새 요소를 추가하여 다양한 적용이 가능하게 만든다.
- ④ 메서드 오버라이딩(method overriding): 부모 클래스 메서드를 자식 클래스에서 재정의
- ⑤ 메서드 오버로딩(method overloading): 같은 이름의 연산자 여러개 정의
- ⑥ 연산자 오버로딩(operator overloading): 새로운 연산자 정의

클래스 - 다형성

다형성(**polymorphism**)은 굉장히 광범위한 개념이다. 정의는 “같은 이름의 것들이 여러 가지 형태로 나타나는 것”을 의미한다.

메서드 오버라이딩, 메서드 오버로딩 전부 다형성의 예시이다.

클래스 - 클래스 관계

클래스 간의 관계를 이해하면 코드와 상황을 더욱 명확하게 이해하는데 도움이 되며, 이러한 관계를 **UML 관계도**를 통해 정리할 수 있다. 이때, 이러한 관계도를 읽고 쓰는 법을 알아야 한다.

클래스 관계는 다음의 4가지가 있다.

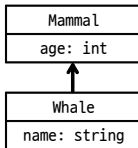
- ① 상속(**inheritance**)
- ② 연관(**association**)
- ③ 집약(**aggregation**)
- ④ 구성(**composition**)

이 중 ③, ④를 합쳐서 집합 관계라고 부른다.

클래스 - 클래스 관계 - 상속

종류	is-a relation
관계도	실선 & 화살표

한 클래스가 다른 클래스를 가져온 뒤, 일부 구현 을 수정하거나 없는 기능을 넣어 변화를 주는 것이다. 무엇인가가 ‘종속’된다는 개념과는 무관하다.



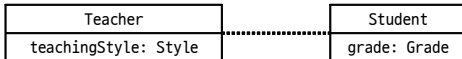
(예시) 포유류와 고래 클래스

- 고래 클래스를 만드려면 포유류 클래스를 상속 해야 한다.
- 고래는 포유류이지만, 포유류가 고래는 아니다.

클래스 - 클래스 관계 - 연관

종류	use-a relation
관계도	점선

서로가 서로에게 소유권이 없기 때문에 종속 관계가 아니다. 서로를 알고 있지만 서로의 생명 주기에는 영향이 없다. 한 인스턴스가 다른 인스턴스를 일방적으로 쓰는 것이 보편적인 구현 방식이기 때문에 has-a relation으로도 표현된다.



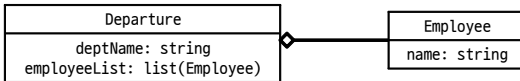
(예시) 선생님과 학생 클래스

- 서로 관련은 있지만 서로에게 속해있지는 않다.
- 서로가 서로의 메서드나 데이터에 영향을 주긴 한다.

클래스 - 클래스 관계 - 집약

종류	has-a relation
관계도	실선 & 빈마름모

연관에 whole-part 관계를 추가했다. 이 관계는 하나의 메인 클래스가 다른 하위 클래스들을 포함하는 관계이다. 메인 클래스가 하위 클래스에 대해 약한 소유권을 가지며, 하위 클래스가 메인 클래스에 종속되지는 않기에 서로의 생명 주기에 영향은 주지 않는다.



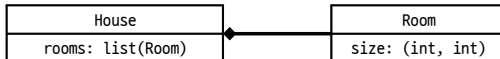
(예시) 회사 부서와 회사원 클래스

- 한 부서 안에 회사원들을 묶을 수는 있지만 완전히 종속된 것은 아니다.
- 부서는 언제든지 옮길 수 있고, 부서가 없어져도 구성원들이 없어지지는 않는다.

클래스 - 클래스 관계 - 구성

종류	owns-a relation
관계도	실선 & 채운마름모

집약과 비슷하지만 메인 클래스가 강한 소유권을 갖는다. 즉, 클래스가 종속되기에 메인 클래스가 하위 클래스의 생명 주기에 영향을 준다. 집약이기도 하기 때문에 has-a relation이기도 하다.



(예시) 집과 방 클래스

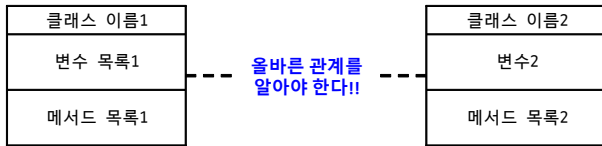
- 집 안에 여러 방이 속해있다.
- 집이 없어지면 마찬가지로 집의 하위 요소였던 방도 동시에 없어진다.

클래스 - UML 관계도

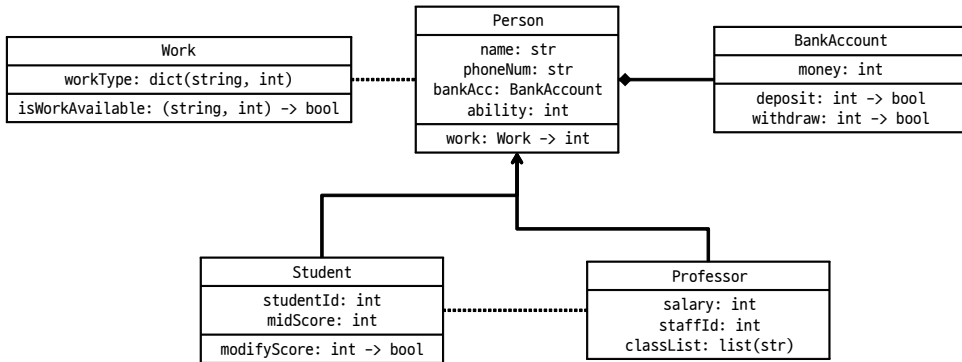
UML 관계도는 아래와 같이 하나의 클래스를 박스 3개로 나누어 그린다. 각각의 박스에는 변수, 메서드가 들어가며 변수나 메서드 항목이 없다면 박스를 생략하여도 된다.

변수와 메서드에는 타입을 명시하여야 한다.

클래스 하나를 그린 다음에는 클래스 관계에 따라 관계도를 그려야 한다. 즉, 클래스 관계를 정확하게 이해하여야 관계도를 올바르게 그릴 수 있다.



클래스 - UML 관계도 예시



Q/A?